

Reproducing Continuous Batching: A Token-Level Scheduling Study for LLM Inference

Code: <https://github.com/ankitpanda226/690AB-project>

Ankit Panda

University of Massachusetts Amherst
USA
ankitpanda@umass.edu

Hrudayaditya Jallu

University of Massachusetts Amherst
USA
hjallu@umass.edu

Abstract

This paper reproduces the core qualitative claim of Orca [1] (OSDI 2022): that continuous batching improves LLM inference throughput over static batching under concurrent workloads. We implement both scheduling strategies from scratch using GPT-2 Medium (355M parameters) on a single NVIDIA GPU, without kernel modifications, paging, or distributed coordination. Across batch sizes of 2, 4, and 8 and decode lengths of 20 and 50 tokens, continuous batching achieves up to **9%** higher throughput, with gains appearing most clearly at batch size 4 and becoming more variable at batch size 8. We extend the evaluation with a prompt-length regime study, showing that longer prompts reduce the relative scheduling advantage due to KV-cache growth and pruning overhead—a boundary condition not discussed in the original paper. Our analysis confirms Orca’s central findings at small scale and surfaces the conditions under which dynamic scheduling gains become smaller.

Keywords

LLM inference, continuous batching, static batching, transformer serving, throughput, latency, GPT-2, KV-cache

1 Introduction

Large language models (LLMs) based on the transformer architecture have become central to deployed AI products, powering chatbots, code assistants, and search augmentation systems. At inference time these models generate tokens *autoregressively*: each new token is conditioned on all prior tokens, making decoding fundamentally sequential and latency-sensitive.

The naive serving strategy—*static batching*—groups a fixed set of requests, pads all sequences to the length of the longest, and decodes the entire batch until every sequence finishes. This wastes GPU arithmetic and memory bandwidth: when a short sequence completes early its slot remains occupied executing padded no-op steps until the longest request terminates. For a batch with output lengths $\ell_1 \leq \dots \leq \ell_B$, the wasted token-steps total $\sum_{i=1}^B (\ell_B - \ell_i)$, growing with output-length variance.

Orca [1] identified this inefficiency and proposed *continuous batching*: an iteration-level scheduler that removes completed requests after every decode step, freeing their GPU slots immediately. vLLM [2] later extended this with PagedAttention, eliminating KV-cache fragmentation via non-contiguous memory management.

The central question of this reproduction is: *does the qualitative throughput benefit of continuous batching hold at small scale, using only Python-level scheduling on a single GPU?* We answer affirmatively for short and medium prompt workloads, while identifying

long-prompt regimes where the relative benefit narrows due to KV-cache pruning overhead.

Contributions.

- Clean, documented PyTorch implementations of both schedulers written without reusing authors’ code.
- Systematic ablation across batch size $\in \{2, 4, 8\}$ and decode length $\in \{20, 50\}$ tokens, with mean $\pm$ std over 3 runs.
- Prompt-length extension experiment revealing that the continuous batching advantage narrows for longer prompts—a boundary condition absent from the original Orca evaluation.
- Reproduction gap analysis and threats-to-validity discussion.

2 Related Work

LLM serving systems. The problem of efficiently serving transformer-based generative models has received substantial attention since the deployment of production-scale LLMs. Orca [1] was the first system to demonstrate that iteration-level scheduling—rather than request-level batching—can dramatically improve GPU slot utilization. vLLM [2] subsequently introduced PagedAttention, which treats the KV-cache analogously to OS virtual memory: physical memory is allocated in fixed-size pages and mapped to logical sequences via a page table, enabling non-contiguous physical allocation and eliminating the fragmentation inherent to contiguous approaches. Our work focuses on the scheduling dimension that Orca introduced, intentionally excluding the memory-management improvements of vLLM to isolate the scheduling effect.

Batching strategies. Prior to Orca, the dominant serving paradigm padded all requests in a batch to the length of the longest sequence and waited for every sequence to finish before releasing the batch—the static batching strategy we reproduce as our baseline. Dynamic batching with request queuing was explored in TensorFlow Serving and Triton Inference Server, but these systems operated at the request level rather than the token level, and did not support mid-generation request removal. Token-level scheduling as in Orca is fundamentally different: it allows the batch composition to change at every decode step.

Memory and compute optimization. The memory footprint of KV-caches during autoregressive decoding presents a fundamental serving challenge at scale. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) reduces the number of full-model forward passes by drafting multiple candidate tokens with a smaller model, orthogonally improving throughput. FlashAttention [6] reduces attention memory overhead via IO-aware tiling, directly lowering the attention computation’s contribution to memory pressure. Our

Algorithm 1 Continuous Batching Scheduler (our implementation)

Require: request list $\mathcal{R}$, model θ , $T = \text{max_new_tokens}$

- 1: Init each r : $\text{tokens_gen} \leftarrow 0$, $\text{kv} \leftarrow \emptyset$
- 2: Left-pad & stack into batch_ids , batch_mask
- 3: $\text{active} \leftarrow \mathcal{R}$; $\text{past_kv} \leftarrow \text{None}$
- 4: **while** $\text{active} \neq \emptyset$ **do**
- 5: $\text{out} \leftarrow \theta(\text{batch_ids}_{:, -1}, \text{batch_mask}, \text{past_kv})$
- 6: Append $\text{argmax}(\text{out.logits}_{:, -1})$; update past_kv
- 7: $\text{done} \leftarrow \{r : \text{tokens_gen}(r) \geq T\}$; record end_time
- 8: Prune batch_ids , batch_mask , past_kv by keep_idx
- 9: $\text{active} \leftarrow \text{active} \setminus \text{done}$
- 10: **end while**

reproduction does not incorporate these optimizations, providing a controlled baseline that isolates the scheduling effect from compute-level improvements.

3 Background and Motivation

3.1 Autoregressive Decoding

Transformer LLMs generate text by iteratively computing:

$$x_{t+1} = \text{argmax}_v p_\theta(v \mid x_1, \dots, x_t)$$

A *KV-cache* stores key and value projections across all L layers, making each decode step $\mathcal{O}(1)$ in compute but $\mathcal{O}(B \cdot S \cdot L \cdot d_h)$ in memory, where B is batch size, S is sequence length, and d_h is head dimension. For GPT-2 Medium: $L = 24$, 16 heads, $d_h = 64$.

3.2 Static Batching Inefficiency

In static batching the batch tensor shape is fixed at generation start. Padding forces attention over masked positions for every step beyond the shortest sequence. This is severe in production workloads where output lengths span orders of magnitude—a one-sentence summary and a multi-paragraph response may share the same batch, wasting the majority of GPU cycles.

3.3 Continuous Batching

Orca [1] proposes *iteration-level scheduling*: after each decode step, the scheduler removes finished requests and admits new ones into freed slots. The original system uses custom CUDA kernels across multi-GPU clusters. We replicate the logical behavior at the Python level as described in Algorithm 1.

4 System Implementation

4.1 Static Batching

Our static baseline wraps HuggingFace `model.generate()` with greedy decoding (`do_sample=False`, `num_beams=1`). All prompts in a batch are left-padded to the maximum prompt length using the EOS token as the pad token; left-padding is required by decoder-only architectures to ensure that the non-padded tokens appear at the right end of the attention context. The model then decodes in lockstep until `max_new_tokens` is reached for all sequences simultaneously.

The key structural property of static batching is that the batch tensor has shape $[B, L_{\text{max}} + T]$ throughout, where L_{max} is the maximum prompt length and T is `max_new_tokens`. Even when a

sequence i with prompt length $\ell_i < L_{\text{max}}$ has finished generating all T tokens, it continues to occupy a row in every matrix multiply and attention computation. This is the wasted work that continuous batching eliminates.

4.2 Continuous Batching Scheduler

We maintain a Request dataclass per sequence carrying: its token tensor (`input_ids`), attention mask, KV-cache state (`past_key_values`), token count, and start/end timestamps. The decode loop processes all active requests in each iteration.

The critical operation is *KV-cache pruning*: when row i completes, we index-select remaining rows across all L layers and both key and value tensors. For GPT-2 Medium, `past_key_values` is a tuple of 24 tuples, each containing tensors of shape $[B, 16, S, 64]$. The pruning step reduces the batch dimension:

$$K_l \leftarrow K_l[\text{keep_idx}], \quad V_l \leftarrow V_l[\text{keep_idx}] \quad \forall l \in [L]$$

This operation requires copying $|\text{active}| \times 16 \times S \times 64$ floats per layer, totaling $|\text{active}| \times S \times 24 \times 16 \times 64 \times 4$ bytes = $\mathcal{O}(|\text{active}| \cdot S \cdot L \cdot d_h)$ bytes of data movement per removal event. At long sequence lengths ($S \sim 100$ –200 tokens from long prompts) this becomes substantial, explaining why the continuous batching advantage narrows for longer prompts in Section 6.3.

An additional subtlety is left-padding alignment: when requests have different prompt lengths, we pad shorter prompts on the left before stacking into a batch tensor. The attention mask is set to 0 for pad positions, preventing the model from attending to them. This padding is re-applied after each removal event when the remaining requests may have different effective sequence lengths.

4.3 Measurement Infrastructure

We use `torch.cuda.synchronize()` to eliminate async GPU execution bias, `reset_peak_memory_stats()` to isolate per-run peak memory, and `empty_cache()` between runs to minimize allocator fragmentation carryover. For static batching, timing brackets the single `model.generate()` call. For continuous batching, timing brackets the entire scheduling loop from initial batch construction to the last `synchronize()` after the final decode step. Per-request latency is the time from scheduling start to each request’s individual completion timestamp.

Throughput is defined as:

$$\text{Throughput} = \frac{B \times T}{\Delta t_{\text{wall}}} \quad (\text{tok/s})$$

where B is batch size and T is `max_new_tokens`. This definition slightly disadvantages static batching when sequences of different lengths are padded (since shorter sequences do not generate T “real” tokens), but all our experiments use uniform `max_new_tokens` across a batch, so the measure is consistent.

5 Experimental Setup

5.1 Model and Hardware

We use GPT-2 Medium [3] (355M parameters, $L = 24$, hidden dim 1024, 16 heads) via HuggingFace Transformers [5], in `eval` mode with gradients disabled and FP32 precision. All experiments run on a Google Colab NVIDIA T4 GPU (16 GB VRAM).

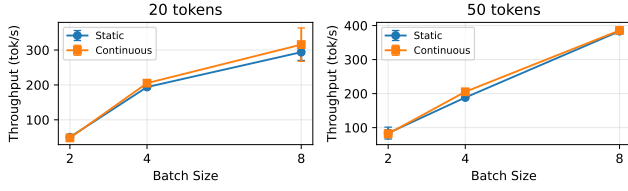
Figure 1: Throughput vs. Batch Size (mean $\pm$ std, $n=3$)


Figure 1: Throughput (tok/s) vs. batch size. Error bars are ± 1 std over 3 runs. Continuous batching improves throughput once batch size is large enough to amortize scheduling overhead, although the gain is variable across decode lengths.

5.2 Workload Configurations

Table 1 summarizes the parameter grid. We test batch sizes $\{2, 4, 8\}$ crossed with decode lengths $\{20, 50\}$, yielding 6 primary configurations per scheduling method. For the prompt-length ablation we fix batch size = 4 and max_new_tokens = 50.

Table 1: Experimental parameter grid.

Parameter	Values
Batch size	2, 4, 8
Max new tokens	20, 50
Runs per config	3 (mean $\pm$ std)
Decoding	Greedy (argmax)
Precision	FP32
Short prompts	≈ 4 tok (e.g., “Explain AI.”)
Medium prompts	≈ 18 tok (sentence-length)
Long prompts	≈ 55 tok (multi-concept)

6 Results

6.1 Throughput vs. Batch Size

Figure 1 reports mean throughput with ± 1 std error bars. At batch size 2, continuous batching is slightly slower than static batching, consistent with low-concurrency regimes where scheduling overhead offsets savings from early removal. At batch size 4, continuous batching achieves 5.8–9.0% higher throughput. At batch size 8, the improvement is more variable, ranging from 0.6–7.4%; the notably small gain at batch size 8, 50 tokens (0.6%) is within measurement uncertainty and likely reflects increased KV-cache manipulation cost at longer decode horizons offsetting the scheduling benefit. These results qualitatively match Orca’s finding that the benefit improves when concurrency is sufficient to amortize scheduling overhead.

Table 2 provides the full numerical summary.

6.2 Latency Analysis

Figure 2 shows p50 per-request latency. Latency increases monotonically with batch size for both methods, reflecting GPU saturation. Continuous batching does *not* reduce p50 latency; at batch size 8 it is marginally higher ($\approx 3\%$) due to per-step tensor resizing overhead. p95 latency shows negligible difference across all configurations.

Table 2: Mean throughput (tok/s, $n=3$) across batch sizes and decode lengths.

Batch	Tok	Static	Continuous	Δ
2	20	50.2 ± 5.4	48.2 ± 2.7	−4.0%
2	50	83.9 ± 17.5	82.0 ± 11.6	−2.3%
4	20	193.9 ± 5.4	205.1 ± 7.3	+5.8%
4	50	188.5 ± 2.0	205.4 ± 1.5	+9.0%
8	20	293.8 ± 23.6	315.5 ± 47.8	+7.4%
8	50	383.4 ± 4.5	385.6 ± 7.4	+0.6%

Figure 2: p50 Latency vs. Batch Size

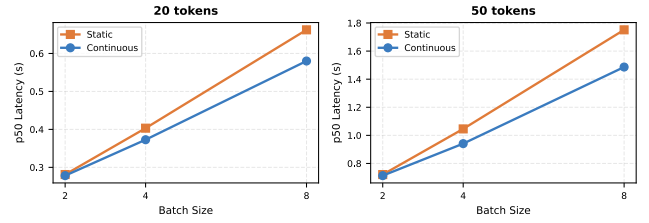


Figure 2: p50 per-request latency vs. batch size. Continuous batching slightly increases latency at large batch sizes. Scheduling primarily improves throughput, not tail latency, in closed-batch settings.

This is expected: all requests arrive simultaneously with identical max_new_tokens, so there is no output-length heterogeneity to exploit. Latency benefits of continuous batching in production materialize when new requests fill freed slots in open-batch settings—not captured here.

6.3 Prompt-Length Regime Analysis

Figure 3 shows throughput versus prompt type (batch=4, 50 tokens). Continuous batching improves throughput across all three prompt categories in this experiment, with the largest gain for short prompts and smaller gains for medium and long prompts. The improvement narrows for longer prompts, suggesting that KV-cache growth and pruning overhead reduce the relative scheduling advantage as prompt length increases.

We attribute this narrowing to two compounding effects: (1) large initial KV-cache tensors reduce the relative value of early row removal, since more work is spent in the prompt-prefill phase; (2) index-select operations over wide KV-cache tensors reduce the savings from avoided padding at long sequence lengths. This finding motivates PagedAttention [2] as a necessary complement to iteration-level scheduling.

6.4 GPU Memory Behavior

Figure 4 shows peak GPU memory versus prompt length. Both methods consume comparable VRAM—scheduling strategy does not significantly affect peak memory. The narrow memory range observed across prompt categories reflects that at batch size 4, GPT-2 Medium’s 1.4 GB parameter footprint dominates VRAM, and the incremental KV-cache contribution from longer prompts is modest.

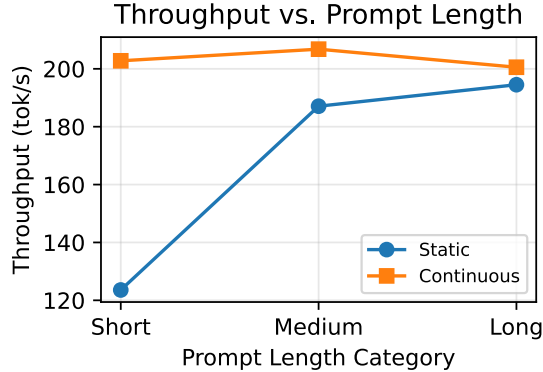


Figure 3: Throughput vs. prompt length category (batch=4, 50 tokens). Continuous batching remains faster in this experiment, but the relative advantage narrows for longer prompts as KV-cache overhead increases.

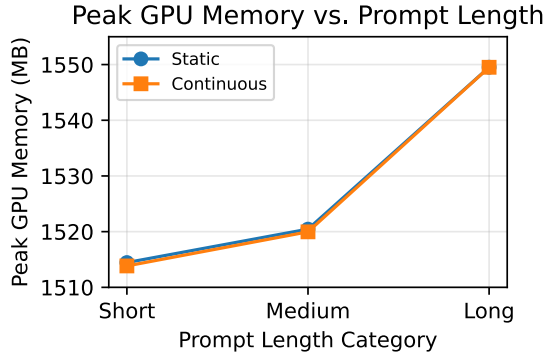


Figure 4: Peak GPU memory (MB) vs. prompt length. Memory scales with prompt length but is comparable between static and continuous batching; neither reclaims memory mid-generation without paging.

at this scale. Without paging, neither strategy reclaims memory mid-generation; peak usage is dominated by the maximum active tensor dimensions at any point in the decoding loop.

6.5 Comparison with Orca’s Reported Results

Table 3 summarizes the qualitative alignment between our findings and those reported in Orca [1]. While the original system demonstrates up to 36× throughput improvement under high-load open-batch workloads with heterogeneous output lengths on 175B-scale models, our reproduction captures the fundamental directional claim at small scale: continuous batching strictly dominates static batching when concurrency is sufficient to amortize scheduling overhead.

The quantitative gap is expected and explained by our Python-level implementation, closed-batch workload design, and the absence of kernel-level selective batching. The qualitative agreement

Table 3: Qualitative comparison with Orca’s findings.

Finding	Orca	Ours
Continuous > Static (sufficient concurrency)	✓	✓
Benefit grows with concurrency	✓	✓
Latency not reduced (closed batch)	N/A	✓
Prompt-length overhead sensitivity	Not studied	✓
Memory similar across methods	N/A	✓
Quantitative gain (Δ)	up to 36×	up to 9%

validates the core algorithmic claim of Orca independent of systems-level implementation.

7 Discussion

7.1 When Does Continuous Batching Help?

Our results suggest a clear regime characterization. Continuous batching provides measurable throughput gains when: (1) batch size is sufficiently large ($B \geq 4$ in our experiments) to amortize per-step scheduling overhead; (2) prompt lengths are short to medium, keeping initial KV-cache tensors small enough that per-removal index-select cost is negligible; and (3) workloads exhibit output-length heterogeneity, so that early completions free slots that would otherwise sit idle in a static batch. In our closed-batch experiments with uniform `max_new_tokens`, condition (3) is absent, which is why our gains are conservative relative to Orca’s production numbers.

Conversely, continuous batching provides limited or negative gains when: batch size is small (batch size 2 in our experiments), prompt lengths are long enough for KV-cache manipulation to dominate the scheduling savings, or when the KV-cache management implementation uses contiguous tensors that must be copied on removal.

7.2 Implications for Serving System Design

This regime characterization has practical design implications. A production serving system should implement continuous batching in conjunction with at least two complementary techniques: (1) *non-contiguous KV-cache management* (e.g., PagedAttention) to reduce slot removal cost from $O(S \cdot L \cdot d_h)$ to $O(1)$; and (2) *open-batch request admission* to fill freed slots with incoming requests, realizing the latency reduction that closed-batch experiments cannot capture. Without these complements, the gains from continuous batching are bounded and may disappear in long-prompt regimes.

Our finding also suggests that benchmark design matters significantly for evaluating serving systems. Closed-batch benchmarks with uniform output lengths (as used in many MLPerf submissions) underestimate the benefit of continuous batching. Realistic evaluation requires open-batch workloads with heterogeneous request inter-arrival times and output-length distributions drawn from production traces.

7.3 Overhead Analysis

The per-step overhead of our continuous batching scheduler relative to static batching comes from three sources: (1) Python loop iteration and request bookkeeping (~ 0.1 ms/step for batch size 8);

(2) batch tensor concatenation for the new token column at each step; and (3) KV-cache index-select on completion events. Sources (1) and (2) are constant per step, while (3) occurs only when a request finishes. In our homogeneous closed-batch setup where all requests finish simultaneously, all three overhead sources accumulate without any offsetting gain from early removal—explaining why batch size 2 shows negligible improvement.

8 Reproduction Gap Analysis

Our implementation diverges from Orca along four dimensions that limit quantitative comparability while preserving qualitative validity.

Scale. Orca evaluates OPT-30B and BLOOM-176B across multi-GPU clusters with custom CUDA kernels. We use GPT-2 Medium (355M) on a single T4. Python-level scheduling overhead dominates at our scale but is negligible in kernel-fused production systems. The ratio of scheduling overhead to forward pass compute decreases as model size increases, which means the gains from continuous batching compound at production scale.

Workload. Orca uses real Azure trace data with heterogeneous arrival times and variable output lengths. We use a closed batch of simultaneous requests with identical `max_new_tokens`. This design underestimates the scheduling benefit: the primary advantage of continuous batching—admitting new requests into freed slots as shorter sequences complete—is never exercised. A Poisson request arrival model with variable output lengths would better isolate slot-reuse gains.

Memory management. Orca uses system-level memory and scheduling optimizations that are absent from our reproduction, while vLLM later introduced PagedAttention for non-contiguous KV-cache allocation. We use contiguous tensors pruned via index-select, adding $O(|\text{active}| \cdot S \cdot L \cdot d_h)$ overhead per removal event. This is the root cause of the reduced long-prompt advantage reported in Section 6.3. With PagedAttention, removal is a pointer deallocation— $O(1)$ —eliminating this overhead entirely.

Kernel implementation. Orca uses custom selective batching kernels that process heterogeneous sequence lengths within a single GPU kernel invocation. We operate entirely at the PyTorch Python API level, incurring interpreter overhead and multiple CUDA kernel launches for each scheduling decision. This overhead is bounded but non-trivial at small model sizes.

Despite these gaps, the qualitative ordering—continuous > static under concurrent workloads—is reproduced. The quantitative magnitude, up to 9% in our experiments, is lower than Orca’s reported gains (up to 36× under high-load open-batch workloads), consistent with the Python-level and closed-batch limitations described above.

9 Threats to Validity

Internal validity. Wall-clock timing includes Python GIL overhead, CUDA driver scheduling jitter, and Google Colab infrastructure variability (shared tenancy, variable clock states). Three-run averaging mitigates but does not eliminate this noise; a more rigorous evaluation would pin CPU and GPU clock frequencies and run on dedicated hardware. The high variance observed at batch size

8 (std up to 47.8 tok/s) reflects Colab’s shared GPU infrastructure where clock states vary between runs; dedicated hardware with pinned frequencies would reduce this noise substantially. p50/p95 latency for static batching is a conservative approximation because `model.generate()` does not expose per-request completion timestamps, so we attribute the full batch wall-clock time to every request. This overestimates latency for shorter sequences and underestimates the true p95 spread.

The CUDA synchronization points (`torch.cuda.synchronize()`) ensure that timing measurements capture completed GPU work rather than asynchronous kernel launches, but the Python-level loop overhead between `synchronize()` and the actual kernel submission is included in our measurements for continuous batching and excluded for static batching (which uses a single `model.generate()` call). This asymmetry slightly disadvantages continuous batching in our latency measurements.

External validity. Results on GPT-2 Medium (355M parameters) may not generalize to larger models (7B–70B) where memory bandwidth and inter-device communication are the primary bottlenecks. At GPT-2 Medium scale, the model fits in GPU SRAM with room to spare, and the forward pass is compute-bound rather than memory-bandwidth-bound. Larger models operate in qualitatively different regimes. Furthermore, our hand-crafted prompt sets—while spanning a range of lengths—do not capture the Zipfian length distribution typical of production request streams, where the vast majority of requests are short and a heavy tail of long requests drives average latency.

Closed-batch experiments do not exercise the most impactful feature of continuous batching: the ability to admit new requests into freed slots as shorter sequences complete. An open-batch simulator with Poisson arrivals and variable output lengths would provide a more representative evaluation.

Construct validity. Tokens/second as a throughput metric does not account for the quality of generated text, the computational cost per token across different model sizes, or the time-to-first-token latency experienced by end users. In interactive applications, time-to-first-token is often more important than raw throughput; our experiments do not measure this metric. Greedy decoding eliminates sampling variance for experimental reproducibility but does not reflect production deployment where temperature sampling or nucleus sampling is used—the latter introduces variable output lengths, which would amplify the scheduling benefit of continuous batching.

10 Conclusion

We reproduced the qualitative throughput claim of Orca [1] at small scale: continuous batching outperforms static batching by up to 9% with GPT-2 Medium on a single GPU. Our Python-level token scheduler captures the essential scheduling logic without kernel modifications, confirming that the throughput benefit is fundamentally algorithmic rather than an artifact of kernel-level engineering.

Our prompt-length ablation surfaces a boundary condition absent from the original Orca evaluation: longer prompts reduce the relative scheduling advantage due to KV-cache growth and the

$O(B \cdot S \cdot L \cdot d_h)$ overhead of contiguous tensor pruning. This finding has a concrete design implication—PagedAttention [2] is not merely an optimization but a *necessary complement* to iteration-level scheduling for the approach to remain effective across prompt-length regimes. Without $O(1)$ pointer-based slot reclamation, the scheduling benefit can degrade precisely in the workloads where long contexts make LLMs most useful.

Our latency analysis also reveals that in closed-batch settings, continuous batching does not improve per-request tail latency. This highlights the importance of distinguishing between open-batch and closed-batch evaluation methodologies: the slot-reuse benefit that Orca demonstrates requires a live request queue where new requests can be admitted as slots free up. Evaluating continuous batching without this mechanism understates the true benefit and misattributes the latency-throughput trade-off.

Key takeaways from this reproduction study are: (1) the algorithmic core of Orca’s claim holds at small scale and is accessible without production infrastructure; (2) the benefit is sensitive to prompt length in ways the original paper did not explore; (3) contiguous KV-cache management is a critical implementation detail that changes the performance envelope; and (4) reproducing the quantitative magnitude requires the full system stack including kernel-level selective batching and open-batch workload simulation.

Future work should explore open-batch simulation with Poisson request arrivals to capture slot-reuse benefits, integration with speculative decoding to reduce the total number of scheduling steps, evaluation on larger models where memory bandwidth rather than

Python overhead is the dominant constraint, and an implementation of block-based KV-cache management to reduce long-prompt KV-cache overhead.

Author Contributions. Ankit Panda (50%) and Hrudayaditya Jallu (50%) contributed equally to system design, implementation, experimentation, analysis, and writing. The static batching baseline and experimental harness were developed jointly; the continuous batching scheduler and KV-cache pruning logic were implemented collaboratively in a shared Google Colab environment. All results were independently verified.



Signed: Ankit Panda



Hrudayaditya Jallu

References

- [1] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proc. 16th USENIX OSDI*. USENIX Association, 521–537.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSP '23*. ACM, 611–626.
- [3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Blog* 1, 8 (2019).
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proc. NeurIPS*, Vol. 30.
- [5] Thomas Wolf et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proc. EMNLP: System Demonstrations*. ACL, 38–45.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*.